

Jaypee University of Engineering and Technology, Guna
B.Tech. (Semester II)
Object Oriented Programming Lab (CS202)
Lab Exercise 6

All programs in this exercise must be designed in a user-friendly manner by taking inputs from the user at runtime. Hardcoded values are not allowed. Ensure that the program prompts the user for input and displays meaningful output messages to enhance user experience.

1. Write a **C++ program** to perform **addition of two complex numbers** using **constructor overloading**. Implement the following:
 - A **default constructor** that initializes the object with **0 + 0i**.
 - A **constructor with one argument** that initializes both real and imaginary parts with the same value.
 - A **constructor with two arguments** that initializes real and imaginary parts separately.
 - A **sum function** that takes two complex numbers as arguments and **returns a new complex number object** containing their sum.
 - A **display function** to print the complex number in the form **a + ib**.
 - Ask the user to input values for two complex numbers at runtime and display their sum.
2. Write a **C++ program** to demonstrate that
 - **new operator calls the constructor** when an object is dynamically allocated.
 - **delete operator calls the destructor** when an object is deleted.

Implementation Details

 - Allow the user to **choose** whether they want to **create or delete** the object dynamically.
 - **Display an error message** if the user attempts to **delete an object before creating it**.
3. Write a **C++ program** that
 - **Dynamically allocates memory** for a matrix using the new operator.
 - **Takes matrix size (rows & columns) and elements** as input from the user at runtime.
 - Passes the **dynamically allocated matrix** to a function.
 - The function returns the **sum of the four corner elements** of the matrix.
 - Frees the allocated memory using delete.
4. A **point in a 2D plane** is represented by coordinates (**x, y**). Implement a **C++ program** that demonstrates the order of **constructor and destructor execution** in an **object-oriented** approach.

Program Requirements:

- **Create a class Point** with:
 - Private data members **x** and **y**.

- A **parameterized constructor** that initializes (x, y) using values provided at runtime.
 - A **destructor** that prints a message when a Point object is destroyed.
 - **Create a class Rectangle with:**
 - Private data members **top-left** and **bottom-right** points, represented as objects of the Point class.
 - A **parameterized constructor** that initializes these points using given coordinates.
 - A **destructor** that prints a message when a Rectangle object is destroyed.
 - **In the main function:**
 - Take **user input** for the coordinates of the top-left and bottom-right corners of the rectangle.
 - Create a Rectangle object and observe the **order in which constructors and destructors are called** when the program runs.
5. Write a **C++ program** to demonstrate that the **constructor** of a **globally declared object** is the **first** function to be called when the program starts, and the **destructor** of the global object is the **last** function to be executed before the program terminates.
6. Write a **C++ program** to demonstrate that **constructors follow function overloading** and can have **default parameters**. Also, show how **default parameters can create issues** in implementing constructor overloading.